

# Documentacion tecnica integral — Jahatelo

**Revision:** 2026-07-19

**Alcance:** aplicacion movil, web publica, panel administrativo, API, datos, integraciones y operacion.

**Fuente:** estructura y codigo de los repositorios locales. No contiene secretos ni valores de produccion.

## 1. Proposito del producto

Jahatelo es una plataforma para descubrir moteles, explorar sus habitaciones, precios, fotos, promociones, menu, ubicacion y reseñas. Opera sobre tres experiencias que comparten el mismo backend:

| Superficie           | Proyecto                   | Usuario principal                           | Funcion                                                                     |
|----------------------|----------------------------|---------------------------------------------|-----------------------------------------------------------------------------|
| Web publica          | web/jahatelo-web           | visitante y usuario registrado              | busqueda, detalle, favoritos, mapas, promos y registro                      |
| Panel administrativo | web/jahatelo-web/app/admin | superadministrador y administrador de motel | alta, edicion, ordenamiento, contenido, comercial, analitica y soporte      |
| App movil            | app/jahatelo-app           | visitante y usuario registrado              | experiencia nativa iOS/Android con el mismo catalogo y acciones principales |

La API Next.js y PostgreSQL son la fuente de verdad. Web y app no deben implementar reglas divergentes sobre planes, moneda, amenities, visibilidad, precios u ordenamiento.

## 2. Arquitectura general



```
| Expo / React Native |
| iOS y Android       |
+-----+
```

Servicios adicionales presentes en el codigo: Expo Push Notifications, SMTP para correo, Upstash Redis para rate limiting, Sentry para monitoreo, Vercel para build/deploy y Cloudinary como configuracion heredada/opcional. Confirmar el proveedor activo de media por entorno antes de cambiarlo.

### 3. Repositorios y estructura

```
projects/jahatelo/
├── app/jahatelo-app/      # Expo / React Native
│   ├── screens/          # pantallas de la app
│   ├── components/       # UI y detalle de motel
│   ├── services/         # cliente API, cache, push, Sentry
│   ├── contexts/, hooks/, constants/ # estado, comportamiento y disen 
│   ├── ios/, android/    # proyectos nativos
│   └── docs/SISTEMA_JAHATELO.md # este documento
├── web/jahatelo-web/     # Next.js 16
│   ├── app/              # web publica, admin y API Route Handlers
│   ├── components/       # componentes compartidos y de dominio
│   ├── contexts/, hooks/, lib/ # estado, utilidades y servicios
│   ├── prisma/schema.prisma # modelo PostgreSQL
│   ├── scripts/          # tareas operativas y guardas de deploy
│   └── vercel.json        # build y cron jobs
```

### 4. Stack tecnologico

#### Backend, web y administracion

- Next.js 16 con React 19 y TypeScript.
- App Router y Route Handlers bajo `app/api`.
- Prisma 6 y PostgreSQL.
- Tailwind CSS para interfaz web.
- Zod para validacion de entradas.
- JWT firmado con `jose`, cookies y/o header `Authorization`.
- S3 mediante AWS SDK para uploads; AWS SNS para SMS OTP/alertas.
- Expo Push para notificaciones moviles.
- Sentry, logger y auditoria para observabilidad.
- Jest, Playwright y ESLint para calidad.

#### Aplicacion movil

- Expo SDK 54, React Native 0.81 y React 19.
- React Navigation, tabs y stack nativo.
- `expo-image`, `expo-location`, `expo-notifications`, `expo-splash-screen`, `expo-av` y Lottie.
- `react-native-maps`, AsyncStorage, NetInfo y Sentry React Native.
- iOS/Android con bundle/package `app.jahatelo.mobile`.

## 5. Modelo de datos

La base es PostgreSQL. Prisma define las entidades y relaciones. La lista siguiente es una vista funcional; el esquema Prisma es el contrato tecnico completo.

| Dominio              | Entidades principales                                                         | Responsabilidad                                            |
|----------------------|-------------------------------------------------------------------------------|------------------------------------------------------------|
| Catalogo             | Motel , RoomType , RoomDayRate , Photo , RoomPhoto                            | motel, habitaciones, tarifas por bloque/dia, fotos y orden |
| Amenities y menu     | Amenity , RoomAmenity , MenuCategory , MenuItem , Product                     | amenities por habitacion y oferta gastronomica             |
| Relacion con usuario | User , Favorite , Review , UserNotificationPreferences , PushToken            | cuentas, favoritos, resenas y preferencias                 |
| Comercial            | Promo , PromoCode , Advertisement , AdAnalytics , HomeBanner                  | promociones, canje, publicidad y medicion                  |
| Operacion            | Schedule , PaymentMethod , SocialLink , PaymentHistory , MotelProspect        | horarios, contactos, cobros y prospectos                   |
| Control              | AuditLog , MotelAnalytics , VisitorEvent , Settings                           | trazabilidad, eventos, visitantes y configuracion          |
| Mensajeria           | WhatsappOtp , ContactMessage , ScheduledNotification , AutoNotificationConfig | OTP, inbox y notificaciones programadas                    |

### Reglas de dominio importantes

1. Los amenities existen por habitacion. Lo que se muestra para un motel es la union de los amenities de sus habitaciones.
2. El orden de habitaciones y fotos es dato persistente; admin, web y app deben respetar el mismo orden.
3. Las portadas separadas `featuredPhotoWeb` (16:9) y `featuredPhotoApp` (4:5) permiten una presentacion correcta por plataforma. Existe una portada de compatibilidad ( `featuredPhoto` ).
4. Los planes soportados son `FREE` , `BASIC` , `GOLD` y `DIAMOND` . La presentacion comercial debe centralizarse; no copiar sus reglas en cada pantalla.
5. Estados de motel: `PENDING` , `APPROVED` , `REJECTED` ; la visibilidad publica exige aprobacion y activacion.
6. Roles: `SUPERADMIN` , `MOTEL_ADMIN` , `USER` . Un administrador de motel se limita a su motel y a los modulos habilitados.

## 6. Web publica

Rutas principales confirmadas:

| Ruta                                              | Funcion                                                         |
|---------------------------------------------------|-----------------------------------------------------------------|
| /                                                 | inicio, destacados, categorias, promociones y publicidad        |
| /search                                           | busqueda y filtros                                              |
| /mapa , /nearby                                   | mapa y cercania                                                 |
| /ciudad/[ciudad] ,<br>/ciudad/[ciudad]/[barrio]   | exploracion geografica                                          |
| /motels/[slug]                                    | ficha del motel: detalles, habitaciones, promos, menu y resenas |
| /mis-favoritos , /perfil                          | experiencia de cuenta                                           |
| /login , /register                                | autenticacion                                                   |
| /registrar-motel                                  | captacion/registro publico                                      |
| /privacidad , /terminos , /soporte ,<br>/contacto | legales y soporte                                               |

Caracteristicas: SEO/JSON-LD, sitemap, PWA, age gate, favoritos, analitica, mapas Google, llamados/WhatsApp, anuncios y manejo de errores.

## 7. Panel administrativo

El panel vive bajo `/admin` y valida autenticacion, rol y permisos de modulo tanto en interfaz como en API.

| Modulo               | Alcance                                                                                                               |
|----------------------|-----------------------------------------------------------------------------------------------------------------------|
| Dashboard            | resumen y acciones rapidas                                                                                            |
| Moteles              | alta, aprobacion, activacion, detalle, contactos, plan, ubicacion, fotos, habitaciones, menu, promos, resenas y orden |
| Amenities            | catalogo de amenities de habitacion                                                                                   |
| Promos               | creacion, vigencia, codigos, canje, busqueda y estados                                                                |
| Banners/Publicidad   | anuncios, placements, variantes web/app y analitica                                                                   |
| Usuarios/Roles       | cuentas, roles y permisos por modulo                                                                                  |
| Comercial/Financiero | prospectos, facturacion, pagos y estado comercial                                                                     |
| Inbox/Notificaciones | mensajes, campanas y programacion push                                                                                |
| Analytics/Auditoria  | indicadores, visitantes, eventos y trazabilidad                                                                       |
| Configuracion        | ajustes operativos publicados                                                                                         |

La pagina de detalle de motel fue modularizada en componentes por dominio. Cambios futuros deben conservar esa division: datos/formularios, fotos, habitaciones, promos, menu, ubicacion y resenas no deben volver a concentrarse en un unico archivo gigante.

## 8. Aplicacion movil

---

Pantallas verificadas: splash y age gate, inicio, lista/busqueda, ciudades, cerca mio, mapa, detalle de motel, favoritos, login/registro, perfil, preferencias de notificacion, historial de promos, contacto y registro de motel.

### Detalle de motel

- Header con portada, acciones de contacto/favorito/compartir y plan.
- Tabs de detalles, habitaciones, reseñas, promos y menu cuando exista contenido.
- Gestos horizontales controlados para tabs sin interferir con galerias verticales/horizontales.
- Galeria de habitacion con previsualizacion fullscreen y swipe entre fotos.
- Los datos se normalizan en `services/motelsApi.js` para tolerar respuestas legadas y mantener fotos/amenities en formato consistente.

### Datos, cache y red

- La app consume `/api/mobile/*` usando `EXPO_PUBLIC_API_URL`.
- Agrega `X-App-Version`; el backend puede responder `426` para exigir actualizacion minima.
- Cache local, vistas recientes, prefetch y soporte de red se concentran en `services/cacheService.js`, `prefetchService.js`, `useNetworkStatus` y `useOnlineRetry`.
- En staging puede usar una compuerta Basic Auth almacenada localmente. No usar esa modalidad para una compilacion de produccion.

### Distribucion nativa

- El release iOS temporal autonomo se construye como `Release` con bundle incluido; no debe depender de Metro.
- La app declara deep links para `jahatelo.com`, permisos de ubicacion/camara/fotos/notificaciones y dominios asociados iOS.
- Todo cambio de plugin, permisos, scheme, bundle/package o `app.json` requiere validacion nativa en iOS y Android.

## 9. API y contratos

---

Las rutas estan bajo `app/api`. Los grupos principales son:

| Prefijo               | Funcion                                                                                                                                                   |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| /api/mobile/*         | contratos consumidos por la app: moteles, detalle, ciudades, mapa, favoritos, resenas, auth y preferencias                                                |
| /api/admin/*          | operaciones autenticadas de administracion: moteles, habitaciones, fotos, amenities, promos, usuarios, financiero, inbox, auditoria, analytics y settings |
| /api/auth/*           | registro, login, sesion, Google y OTP/WhatsApp segun flujo                                                                                                |
| /api/public/*         | registro/captacion publica de moteles                                                                                                                     |
| /api/advertisements/* | lectura y tracking de anuncios                                                                                                                            |
| /api/analytics/*      | eventos de catalogo y visitantes                                                                                                                          |
| /api/upload/*         | upload general y firma/flujo S3                                                                                                                           |
| /api/notifications/*  | programacion y consulta de notificaciones                                                                                                                 |
| /api/cron/*           | tareas programadas protegidas                                                                                                                             |
| /api/health           | health check                                                                                                                                              |

Reglas de contrato:

1. Validar payloads de borde con Zod antes de persistir.
2. Conservar compatibilidad de respuestas movil mientras haya versiones antiguas soportadas.
3. Responder errores estables y no filtrar secretos, SQL ni stack traces.
4. Cualquier campo nuevo de API debe evaluarse en web, admin, app, Prisma y migracion si aplica.

## 10. Seguridad

- JWT HS256, expiracion de siete dias y verificacion de token en servidor.
- Control de rol y permisos de modulo con `requireAdminAccess`.
- Middleware: HTTPS en produccion, CORS permitido, CSRF para operaciones web basadas en cookies, rate limiting y proteccion de staging.
- Upstash Redis es el backend preferido del rate limit; existe fallback en memoria para edge/desarrollo.
- Sanitizacion de texto/HTML, Zod, control de archivos y limites de upload.
- Secrets solamente en variables de entorno. Nunca en Git, PDF, logs, capturas o cliente.
- Cron endpoints deben requerir `CRON_SECRET` /autorizacion de Vercel.

## 11. Integraciones externas

| Servicio     | Uso                    | Variables/configuracion relevantes                            |
|--------------|------------------------|---------------------------------------------------------------|
| PostgreSQL   | datos transaccionales  | <code>DATABASE_URL</code> , <code>DIRECT_URL</code>           |
| Vercel       | hosting, builds y cron | branch de produccion y variables del proyecto                 |
| AWS S3       | media subida           | credenciales AWS y bucket/configuracion de storage            |
| AWS SNS      | SMS OTP y alertas      | region, sender, secretos AWS                                  |
| Expo         | build y push           | <code>EXPO_PUBLIC_API_URL</code> , proyecto EAS y tokens push |
| Google Maps  | geocoding y mapas      | claves servidor/publica                                       |
| Google OAuth | login                  | client IDs web/iOS/Android/Expo                               |
| SMTP         | email y verificacion   | host, puerto, usuario, password                               |
| Sentry       | errores y trazas       | DSN servidor/cliente/movil                                    |
| Upstash      | rate limiting          | URL y token REST                                              |

## 12. Media e imagenes

1. Las imagenes de contenido se almacenan en servicio de media, no en Git.
2. La subida debe validar tipo, tamano, autorizacion y URL resultante.
3. Mantener variantes que respondan a plataforma: web horizontal y app vertical donde corresponda.
4. Toda lista de fotos requiere orden estable y fallback seguro.
5. La web debe usar el componente de imagen apropiado; la app usa sus componentes nativos para carga/cache.
6. Antes de borrar media verificar referencias en motel, habitacion, promo, banner y almacenamiento remoto.

## 13. Notificaciones y comunicaciones

- La app registra tokens Expo y preferencias por usuario/dispositivo.
- El backend selecciona destinatarios por usuarios, rol, favoritos de motel o invitados, respetando preferencias segun categoria.
- Las campanas programadas se guardan en `ScheduledNotification` y el cron `process-notifications` las procesa diariamente a las 03:00 UTC configurada por Vercel.
- Limpieza de datos y codigos promo tiene crons separados a las 04:00 y 05:00.
- Mensajes de contacto alimentan el inbox y pueden disparar alertas a administracion.

## 14. Analitica y auditoria

- Eventos de motel: vista, click telefono, WhatsApp, mapa, web y favoritos.

- Eventos de publicidad: vista y click por placement.
- Eventos de visitantes y analitica administrativa.
- `AuditLog` registra acciones sensibles; todo cambio administrativo relevante debe conservar actor, recurso, accion y fecha.
- Sentry complementa trazas de fallas. No usarlo para datos sensibles.

## 15. Variables de entorno

Usar `.env.example` como contrato. Principales grupos:

```
DATABASE_URL, DIRECT_URL
JWT_SECRET, OTP_SECRET, EMAIL_VERIFICATION_SECRET, CRON_SECRET
AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_SNS_*
SMTP_*, EMAIL_FROM_*
GOOGLE_MAPS_API_KEY, NEXT_PUBLIC_GOOGLE_MAPS_API_KEY
NEXT_PUBLIC_GOOGLE_CLIENT_ID
NEXT_PUBLIC_APP_URL, EXPO_PUBLIC_API_URL
SENTRY_DSN, NEXT_PUBLIC_SENTRY_DSN
UPSTASH_REDIS_REST_URL, UPSTASH_REDIS_REST_TOKEN
STAGING_GATE_*, ALLOWED_PRODUCTION_BRANCH
```

Nunca copiar valores reales a documentacion. Cualquier variable `NEXT_PUBLIC_*` o `EXPO_PUBLIC_*` es visible en el cliente y no debe contener secretos.

## 16. Desarrollo, calidad y despliegue

### Comandos frecuentes

```
# web
cd '/Users/jota/Desktop/AKAHATA STUDIO/projects/jahatelo/web/jahatelo-web'
npm run lint
npx tsc --noEmit --incremental false
npm test -- --runInBand
npm run build

# app
cd '/Users/jota/Desktop/AKAHATA STUDIO/projects/jahatelo/app/jahatelo-app'
npm run lint
npm run typecheck
npm test -- --runInBand
```

### Base de datos

```
cd '/Users/jota/Desktop/AKAHATA STUDIO/projects/jahatelo/web/jahatelo-web'
npx prisma generate
npx prisma migrate dev --skip-seed # desarrollo, con revision previa
npx prisma migrate deploy # entorno de deploy
```



No ejecutar migraciones destructivas ni restauraciones de base sin backup verificable y aprobacion. Un build puede requerir acceso a la base por la generacion/ejecucion de Prisma; diferenciar una dependencia de entorno de un error de compilacion.

## Deploy

1. Revisar `git status` y diff.
2. Validar lint, TypeScript y pruebas del alcance.
3. Confirmar cuenta, repositorio, rama y proyecto Vercel correctos.
4. Confirmar variables de entorno y migraciones requeridas.
5. El build de Vercel protege produccion mediante `scripts/enforce-production-branch.js` ; la rama permitida se controla con `ALLOWED_PRODUCTION_BRANCH` .
6. No hacer commit, push, merge o deploy sin confirmacion explicita del responsable.

## 17. Runbook de incidencias

| Sintoma                | Primeras verificaciones                                                                        |
|------------------------|------------------------------------------------------------------------------------------------|
| Web 500                | logs de Vercel/Sentry, variables, Prisma/DB, endpoint concreto                                 |
| App sin datos          | <code>EXPO_PUBLIC_API_URL</code> , red, endpoint <code>/api/mobile</code> , version minima 426 |
| App abre launcher Expo | se instalo Development Build; crear Release con bundle nativo                                  |
| Fotos faltantes        | URL guardada, permisos/bucket, referencias y variante web/app                                  |
| Mapa incorrecto        | <code>mapUrl</code> , latitud/longitud, geocoding y clave Google Maps                          |
| Push no llega          | token activo, permisos, preferencias, payload Expo, cron y logs                                |
| Admin sin acceso       | token, rol, permisos de modulo, motel asignado y middleware                                    |
| Build falla por DB     | validar variables/alcance de Prisma; no confundir con error TypeScript                         |

## 18. Normas de evolucion

1. Todo cambio funcional debe evaluarse para web publica, admin y app.
2. Reutilizar contratos y reglas de dominio; no duplicar logica de planes, precios, estado u orden.
3. Separar componentes por dominio y mantener TypeScript estricto en web.
4. Cambiar APIs y Prisma con migracion, validacion y compatibilidad planificada.
5. Probar casos exitosos y de error, no solo la pantalla principal.
6. Documentar integraciones nuevas, variables, permisos, cron y plan de rollback.
7. Los archivos de recovery o backups temporales no son parte del producto: usar Git, backups de DB y storage documentado.

## 19. Estado de verificacion de esta revision

- App: lint y pruebas locales aprobadas (6 pruebas).

- Web: TypeScript y pruebas locales aprobadas (40 pruebas).
  - Web: lint sin errores; persisten advertencias de optimizacion/estructura que se siguen corrigiendo gradualmente.
  - No se ejecuta deploy, migracion ni cambio de secretos como parte de esta documentacion.
- 

**Documento vivo.** Actualizar esta guia cuando cambien contratos de API, entidades Prisma, proveedores externos, permisos nativos, cron jobs o proceso de despliegue.